

FLASHCHAT

End-to-End Encryption (E2EE) System Architecture

A Detailed Analysis of Hybrid Cryptography, Client-Side Key Management, and Trustless Relay Protocols.

DOCUMENT METADATA

System Name:	FlashChat Secure Messenger
Implementation Tech:	Web Crypto API (RSA-OAEP 2048, AES-GCM 256)
Status:	Production Ready (Integrated with AuthContext & ChatsContext)
Date Generated:	June 2026

ABSTRACT

This document details the architectural design and implementation of the client-side End-to-End Encryption (E2EE) system implemented in the FlashChat messaging application. FlashChat employs a robust, zero-knowledge hybrid cryptographic system that integrates the Web Crypto API on the client side with a trustless MongoDB database on the backend.

By encrypting message contents before transmission and delegating key exchange to asymmetric RSA key pairs, the system guarantees absolute privacy. The intermediate server serves purely as a socket relay and ciphertext archive, with no technical capability to decrypt the message content. This analysis provides code demonstrations, sequence workflows, and a critical evaluation of local key storage and potential attack surfaces.

1. CORE CRYPTOGRAPHIC COMPONENTS

To satisfy both high performance and secure key distribution, FlashChat uses a hybrid encryption approach. Cryptographic operations are performed natively in the browser via the Web Crypto API (`window.crypto.subtle`), ensuring that plaintext messages and private keys never travel over the network.

1.1 Asymmetric Cryptography: RSA-OAEP 2048-bit

Asymmetric cryptography is used for secure key establishment between unacquainted clients. During user login, the client generates a unique 2048-bit RSA key pair. RSA-OAEP (Optimal Asymmetric Encryption Padding) with a SHA-256 hash algorithm is selected for encryption and decryption.

- **RSA Public Key:** Exported as a JSON Web Key (JWK) string and uploaded to the server database. It is visible to other users to encrypt symmetric keys intended for the user.
- **RSA Private Key:** Stored locally in the user's browser (`localStorage`) and never uploaded to the server. It is retrieved locally to decrypt incoming symmetric keys.

1.2 Symmetric Cryptography: AES-GCM 256-bit

Symmetric cryptography is utilized for encrypting the actual message payload. Because RSA is computationally expensive and restricted by payload size limits, a fresh 256-bit AES (Advanced Encryption Standard) key is generated in Galois/Counter Mode (GCM) for every single message sent.

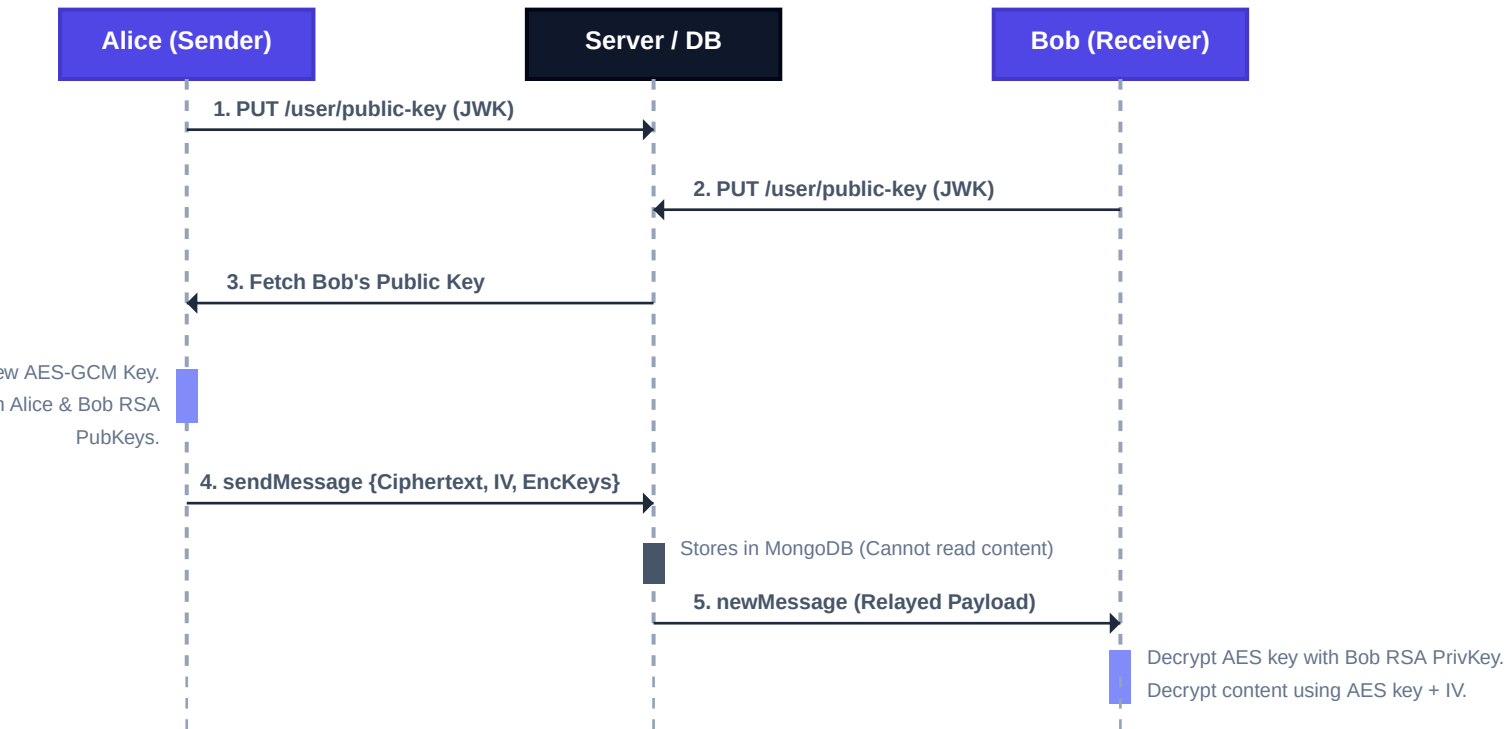
- **Galois/Counter Mode (GCM):** Provides Authenticated Encryption with Associated Data (AEAD). It ensures both confidentiality (encryption) and integrity (tamper detection) of the message content.
- **Initialization Vector (IV):** A random 12-byte buffer generated for each message. The IV is sent in the clear alongside the ciphertext; it prevents replay attacks and ensures identical plaintexts yield different ciphertexts.

1.3 The Hybrid Encryption Model

Combining these two techniques allows FlashChat to achieve both efficiency and scalability. The plaintext message is encrypted with the symmetric AES key, which is highly efficient. The AES key is then encrypted with each recipient's asymmetric public key. As a result, only a tiny symmetric key is encrypted using RSA, saving CPU cycles. In a 1-on-1 chat, the AES key is encrypted twice: once for the sender (to allow decryption of their sent history) and once for the receiver.

2. THE END-TO-END ENCRYPTION FLOW

The E2EE flow consists of three major phases: Key Setup, Sending (Encryption), and Receiving (Decryption). The sequence diagram below shows the runtime interactions between Alice (Sender), the Server, and Bob (Receiver).



2.1 Step-by-Step Sequence Breakdown

- **Key Setup:** Alice and Bob generate RSA key pairs. Alice's private key goes to local storage, and her public key is sent to the server. Bob does the same.
- **Key Exchange:** When Alice opens a chat with Bob, she fetches Bob's public key from the server (which is included in the chat participant's metadata).
- **Message Dispatch:** Alice's client generates a

random AES-GCM key, encrypts the message content, and then encrypts this AES key with her own public key and Bob's public key. The payload is sent to the server.

- **Server Storage:** The server stores the ciphertext, the IV, and the encrypted keys list in MongoDB, then relays the payload to Bob via Socket.io.

- **Message Retrieval:** Bob's client extracts the encrypted AES key corresponding to his userId, decrypts it with his local RSA private key, and decrypts the ciphertext using the AES key and IV.

3. FRONTEND CRYPTOGRAPHIC IMPLEMENTATION

The cryptographic functions reside in `client/src/utils/crypto.js`. Below are the key implementations of key generation, encryption, and decryption.

3.1 Key Pair Generation

```
export async function generateE2EEKeyPair() {
  const keyPair = await window.crypto.subtle.generateKey(
    {
      name: "RSA-OAEP",
      modulusLength: 2048,
      publicExponent: new Uint8Array([1, 0, 1]),
      hash: "SHA-256",
    },
    true,
    ["encrypt", "decrypt"]
  );

  const exportedPublic = await window.crypto.subtle.exportKey("jwk", keyPair.publicKey);
  const exportedPrivate = await window.crypto.subtle.exportKey("jwk", keyPair.privateKey);

  return {
    publicKeyString: JSON.stringify(exportedPublic),
    privateKeyString: JSON.stringify(exportedPrivate)
  };
}
```

3.2 Message Encryption (Hybrid)

```
export async function encryptMessage(text, senderId, senderPublicKeyJwk, receiverId,
receiverPublicKeyJwk) {
  // 1. Generate random AES symmetric key
  const aesKey = await window.crypto.subtle.generateKey(
    { name: "AES-GCM", length: 256 },
    true,
    ["encrypt", "decrypt"]
  );

  // 2. Encrypt text content using AES-GCM
  const iv = window.crypto.getRandomValues(new Uint8Array(12));
  const encodedText = new TextEncoder().encode(text);
  const encryptedContentBuffer = await window.crypto.subtle.encrypt(
    { name: "AES-GCM", iv },
    aesKey,
    encodedText
  );

  // 3. Export raw AES key to encrypt it with recipients' RSA public keys
  const rawAesKey = await window.crypto.subtle.exportKey("raw", aesKey);
  const encryptedKeys = [];

  // Encrypt for Alice (sender)
  const senderPubKey = await importPublicKey(senderPublicKeyJwk);
  const senderEncKey = await window.crypto.subtle.encrypt({ name: "RSA-OAEP" }, senderPubKey,
rawAesKey);
  encryptedKeys.push({ userId: senderId, key: arrayBufferToBase64(senderEncKey) });

  // Encrypt for Bob (receiver)
  const receiverPubKey = await importPublicKey(receiverPublicKeyJwk);
  const receiverEncKey = await window.crypto.subtle.encrypt({ name: "RSA-OAEP" },
receiverPubKey, rawAesKey);
  encryptedKeys.push({ userId: receiverId, key: arrayBufferToBase64(receiverEncKey) });

  return {
    ciphertext: arrayBufferToBase64(encryptedContentBuffer),
    encryption: { isEncrypted: true, iv: arrayBufferToBase64(iv), encryptedKeys }
  };
}
```

4. DECRYPTION & DATABASE SCHEMAS

4.1 Message Decryption

```
export async function decryptMessage(encryptedMsg, currentUserId, currentUsername) {
  const { iv: ivBase64, encryptedKeys } = encryptedMsg.encryptedKeys;

  // 1. Locate current user's encrypted version of the AES key
  const userKeyObj = encryptedKeys.find(k => k.userId?.toString() ===
currentUserId?.toString());

  // 2. Import local private key from localStorage
  const privateKeyJwkStr = localStorage.getItem(`e2ee_private_key_${currentUsername}`);
  const privateKey = await importPrivateKey(privateKeyJwkStr);

  // 3. Decrypt the AES raw key using the RSA private key
  const rawAesKey = await window.crypto.subtle.decrypt(
    { name: "RSA-OAEP" },
    privateKey,
    base64ToArrayBuffer(userKeyObj.key)
  );

  // 4. Import the AES key and decrypt the ciphertext content
  const aesKey = await window.crypto.subtle.importKey("raw", rawAesKey, { name: "AES-GCM" },
true, ["decrypt"]);
  const decryptedBuffer = await window.crypto.subtle.decrypt(
    { name: "AES-GCM", iv: base64ToArrayBuffer(ivBase64) },
    aesKey,
    base64ToArrayBuffer(encryptedMsg.content)
  );

  return new TextDecoder().decode(decryptedBuffer);
}
```

4.2 Zero-Knowledge Mongoose Schemas

The backend stores the encrypted payload inside the MongoDB database. The User model keeps the public key (JWK format), and the Message model stores the ciphertext and encryption headers. The server never handles plaintext.

```
// user.js Schema Field
publicKey: {
  type: String,
  required: false
}

// message.js Schema Fields
content: {
  type: String, // Stores base64 AES-GCM ciphertext
  trim: true
},
encryption: {
  isEncrypted: { type: Boolean, default: false },
  iv: { type: String }, // Base64 Initialization Vector
  encryptedKeys: [
    {
      userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
      key: { type: String } // Base64 RSA-OAEP encrypted AES key
    }
  ]
}
```


5. SECURITY AND VULNERABILITY ANALYSIS

While the hybrid encryption system protects message confidentiality on the transit and storage layers, several real-world security considerations arise from this browser-based cryptographic architecture.

5.1 Private Key Storage and XSS Vulnerabilities

The RSA private key is stored inside the browser's `localStorage`. The primary threat here is Cross-Site Scripting (XSS). If an attacker manages to inject malicious JavaScript into the client application, they can access `localStorage`, read the private key, and exfiltrate it. Once compromised, the attacker can decrypt all past and future messages.

Remediation: Store the private key in an IndexedDB with security policies, or derive it on-demand from the user's password using PBKDF2 (Password-Based Key Derivation Function 2) without saving it on disk in cleartext.

5.2 Lack of Perfect Forward Secrecy (PFS)

In FlashChat, the same RSA public/private key pair is used indefinitely for a user. If Bob's private key is compromised today, an attacker who has been sniffing the network and archiving ciphertext can decrypt every message Bob has ever received. Modern secure protocols (such as Signal) employ the Double Ratchet Algorithm, combining ephemeral Diffie-Hellman handshakes to generate rotating, disposable keys for each message block. This guarantees that compromise of a key today does not compromise past history.

5.3 Man-in-the-Middle (MitM) Attacks via Key Spoofing

Because the server registers and distributes public keys, the system relies on the trust of the server. A malicious or compromised server could perform a Man-in-the-Middle attack. When Alice asks for Bob's public key, the server can supply a public key generated by the server itself. Alice encrypts the AES key with the server's public key. The server decrypts it, reads the message, encrypts it again with Bob's actual public key, and forwards it. Neither Alice nor Bob would notice.

Remediation: Implement out-of-band identity verification (e.g., verifying 'Safety Numbers' or visual QR codes containing public key digests outside the chat channel).

5.4 Authentication Session Decoupling

The cryptographic keys are tied to the username. In `AuthContext.jsx`, if the server cookie state is active but local storage keys are missing, the client automatically resets and registers a new key pair. This is convenient but allows an attacker who intercepts a session cookie to register a new public key and receive messages meant for the user, triggering a 'key changed' event. The application should warn users if a contact's public key has changed unexpectedly.

